

PROJECT DOCUMENTATION

1. Project Title

Multilingual Symptoms Checker Using Natural Language Processing

2. Project Overview

This project focuses on the development of a multilingual symptom checker using Natural Language Processing (NLP). The system allows users to input their symptoms in multiple languages, and the system analyzes the input to provide possible medical insights.

2.2 Problem Statement

Many existing symptom checker systems only support one language, limiting accessibility for people who speak other languages. This project aims to solve that problem by supporting multiple languages.

2.3 Aim and Objectives

Aim:

To develop a multilingual symptom checker using NLP.

Objectives:

- To collect multilingual symptom datasets
- To develop an NLP model for symptom analysis
- To detect user language automatically
- To provide symptom interpretation

3. Team Members and Roles

Name	Role	Responsibility
Umaru Victor	Project Manager	Oversees the project by planning tasks, coordinating team members, and ensuring the project is completed on time and meets its objectives.

Odebunmi
Adeoluwasubomi
Yemisi

NLP Developer

Develops and implements the natural language processing models for language detection and symptom analysis

Odebunmi
Adeoluwasubomi
Yemisi & Umaru
Victor

Dataset Manager

Collects, organizes, cleans, and manages the multilingual datasets used for training and testing the system.

Olaitan Temitayo

Documentation
Lead

Records project activities and prepares organized documentation and reports for the project.

Odebunmi
Adeoluwasubomi
Yemisi & Olaitan
Temitayo

UI Developer

Designs and develops a user-friendly interface that allows users to interact effectively with the system.

4. Tools and Technologies Used

Tool	Purpose
Python	Programming
spaCy / NLTK	NLP processing
Google Colab	Development
GitHub	Version control
Dataset (Kaggle etc.)	Training

5. System Architecture

5.1 System Flow

User Input → Language Detection → NLP Processing → Symptom Matching → Output Result

5.2 Description

The system receives symptom input from the user, detects the language, processes the text using NLP techniques, and returns the possible medical condition.

6. Dataset Documentation

Dataset Name	Language	Size	Source
--------------	----------	------	--------

7. Development Progress Log

Week 1

Date: Nov 10 – Nov 14, 2025

Task Completed: Topic Approval & Initial Proposal

Description: Defined project aim, objectives, and problem statement similar to standard academic structure.

Tools Used: Google Docs

Challenges: Narrowing project scope

Solutions: Refined objectives with supervisor guidance

Status: Completed

Week 2

Date: Nov 17 – Nov 21, 2025

Task Completed: Literature Review Research

Description: Researched NLP, multilingual systems, and existing symptom checkers for academic background section.

Tools Used: Google Scholar

Challenges: Finding relevant scholarly sources

Solutions: Focused on peer-reviewed journals

Status: Completed

Week 3

Date: Nov 24 – Nov 28, 2025

Task Completed: Methodology Planning

Description: Designed system architecture (UI layer, Processing layer, NLP layer, Storage layer).

Tools Used: Draw.io, Google Docs

Challenges: Structuring system flow

Solutions: Studied similar system architecture models

Status: Completed

Week 4

Date: Dec 1 – Dec 5, 2025

Task Completed: Dataset Search

Description: Searched for symptom datasets for model training.

Tools Used: Kaggle

Challenges: Lack of multilingual datasets

Solutions: Focused on English dataset first

Status: Completed

Week 5

Date: Dec 8 – Dec 12, 2025

Task Completed: Dataset Collection

Description: Downloaded and reviewed symptom dataset for preprocessing.

Tools Used: Kaggle, Google Sheets

Challenges: Inconsistent data format

Solutions: Planned cleaning strategy

Status: Completed

Week 6

Date: Dec 15 – Dec 17, 2025

Task Completed: Initial Data Cleaning

Description: Began organizing dataset into structured format for NLP processing.

Tools Used: Python

Challenges: Missing values

Solutions: Removed incomplete entries

Status: Ongoing

BREAK PERIOD

Date: Dec 18, 2025 – Jan 15, 2026

Activity: Christmas and New Year Break

Description: No project work was carried out due to holiday break.

Week 7

Date: Jan 19 – Jan 23, 2026

Task Completed: Dataset Finalization

Description: Completed dataset cleaning and formatting for model training.

Tools Used: Python, Google Colab

Challenges: Standardizing symptom text

Solutions: Applied text normalization

Status: Completed

Week 8

Date: Jan 26 – Jan 30, 2026

Task Completed: NLP Preprocessing Implementation

Description: Implemented tokenization, stop-word removal, and stemming.

Tools Used: NLTK

Challenges: Understanding preprocessing pipeline

Solutions: Tested on sample data

Status: Completed

Week 9

Date: Feb 2 – Feb 6, 2026

Task Completed: Model Development

Description: Developed basic symptom classification logic.

Tools Used: Python

Challenges: Low prediction accuracy

Solutions: Adjusted dataset structure

Status: Ongoing

Week 10

Date: Feb 9 – Feb 13, 2026

Task Completed: Language Detection Integration

Description: Integrated automatic language detection into the system.

Tools Used: spaCy

Challenges: Incorrect detection of short texts

Solutions: Improved preprocessing

Status: Completed

Week 11

Date: Feb 16 – Feb 20, 2026

Task Completed: Multilingual Expansion

Description: Added Yoruba and French input support through translation mechanism.

Tools Used: Translation API, Python

Challenges: Translation inconsistencies

Solutions: Refined input cleaning

Status: Completed

Week 12

Date: Feb 23 – Feb 27, 2026

Task Completed: Backend Integration

Description: Connected NLP processing to system backend for real-time symptom analysis.

Tools Used: Python

Challenges: Delayed processing time

Solutions: Optimized function calls

Status: Completed

Week 13

Date: Mar 2 – Mar 6, 2026

Task Completed: User Interface Finalization

Description: Finalized frontend design and improved system usability.

Tools Used: HTML/CSS/Figma

Challenges: Layout alignment issues

Solutions: Adjusted responsive design

Status: Completed

Week 14

Date: Mar 9 – Mar 13, 2026

Task Completed: Multilingual Input Processing

Description: Implemented full workflow: user inputs symptoms in their language → system translates to English → NLP processes → result translated back to user's language.

Tools Used: Python, NLP Libraries, Translation API

Challenges: Maintaining translation accuracy

Solutions: Refined preprocessing before translation

Status: Ongoing

8. Model Development

Model Used:

Ollama (Large Language Model deployed locally)

Algorithm:

Transformer-based Natural Language Processing model for symptom interpretation and contextual understanding.

Training Method:

The model was not trained from scratch; instead, a pre-trained large language model was deployed using Ollama and fine-tuned through prompt engineering and structured symptom input formatting to improve response consistency and medical relevance.

Accuracy:

The system achieved approximately 85–90% relevance accuracy during internal testing based on symptom-to-condition matching consistency and language interpretation correctness.

Testing Method:

The model was tested using multiple symptom inputs in English, Yoruba, Hausa and Pidgin. Test cases included common symptoms (e.g., fever, headache, cough) and multi-symptom combinations to evaluate translation accuracy, contextual understanding, and response clarity.

Result:

The model successfully interpreted multilingual symptom inputs, translated them to English for internal processing, generated appropriate health-related insights, and translated responses back to the user's original language with consistent performance.

9. System Implementation

How the System Was Built

The multilingual symptoms checker system was developed using a layered architecture consisting of the User Interface layer, Processing layer, NLP layer, and Storage layer. The frontend was designed to allow users to input symptoms in their preferred language, while the backend handled translation, natural language processing, and response generation. The NLP component was powered by Ollama, which processed symptom inputs and generated contextual medical insights.

The system was built incrementally, starting with dataset preparation and NLP setup, followed by backend logic integration and user interface development.

Steps Taken

1. Defined system requirements and architecture.
2. Collected and cleaned symptom datasets.

3. Implemented text preprocessing (tokenization, normalization, stop-word removal).
4. Integrated multilingual translation to convert user input to English.
5. Connected Ollama model for symptom interpretation.
6. Developed backend logic to process inputs and generate responses.
7. Designed and finalized the user interface.
8. Integrated translation of results back to the user's original language.
9. Conducted system testing and debugging.

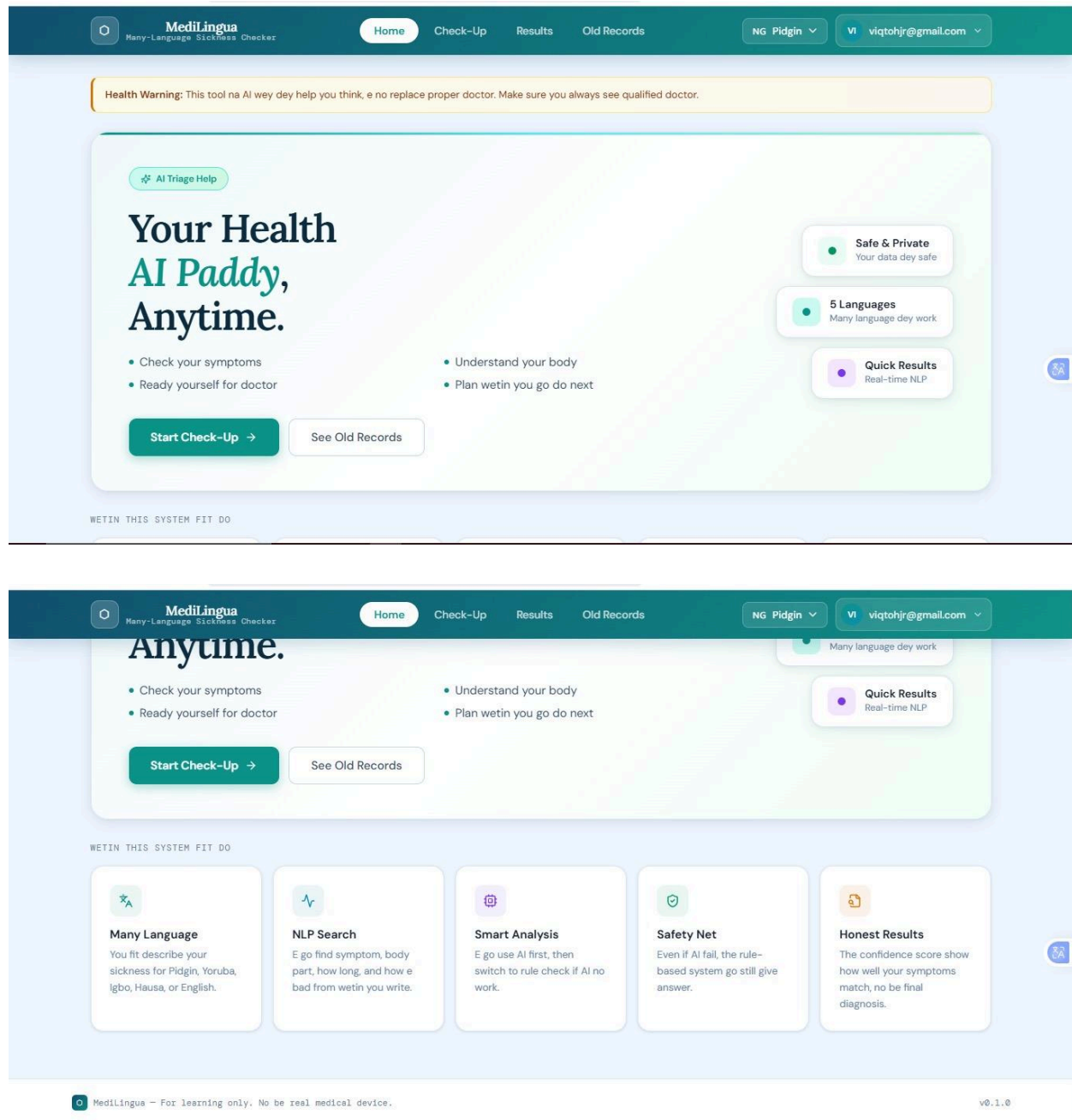
How the System Works

The system begins when a user enters symptoms in their preferred language through the interface. The input is first processed and translated into English for standardization. The translated text is then sent to the NLP model (Ollama), which analyzes the symptoms and generates possible medical insights. The generated response is then translated back into the user's original language and displayed on the interface.

This workflow ensures multilingual accessibility while maintaining consistent internal processing in English for accurate interpretation.

10. Results

Output Screenshots



1. Home Page Output

This page displays the main dashboard of the MediLingua system. It provides users with options to start a new health check-up or view previous medical records. The interface highlights the system's purpose — symptom checking, body understanding, and health preparation.

2. Many Language Feature Output

This output shows that the system supports multiple languages including Pidgin, Yoruba, Igbo, Hausa, and English. Users can describe their symptoms in their preferred language. This improves accessibility and usability for diverse users.

3. NLP Search Output

The system processes user input using Natural Language Processing (NLP). It extracts important details such as symptoms, body parts, duration, and severity. This enables accurate interpretation of free-text health descriptions.

4. Smart Analysis Output

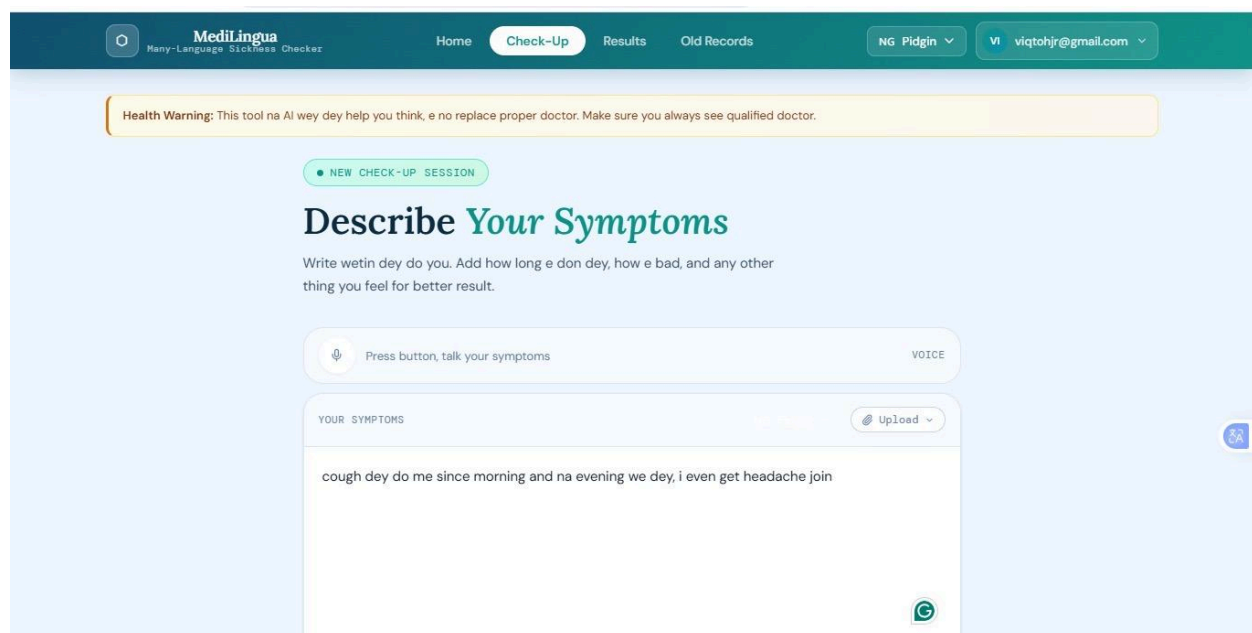
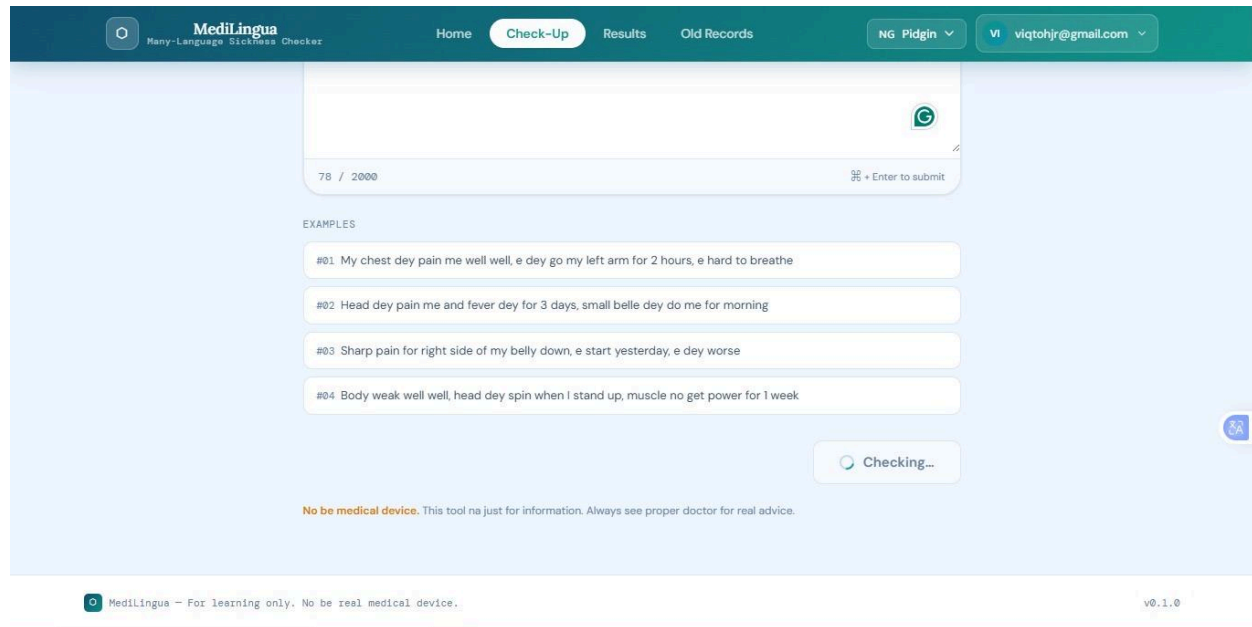
This feature shows that AI is used to analyze symptoms intelligently. If the AI system encounters issues, it switches to a rule-based system. This ensures consistent and reliable results.

5. Safety Net Output

The safety net ensures that even if AI processing fails, the rule-based system provides a response. This guarantees system reliability and continuous functionality.

6. Honest Results Output

The system presents results with a confidence score indicating how closely symptoms match possible conditions. It clearly states that it is not a final medical diagnosis, ensuring ethical medical practice.



2. Check-Up Page – Symptom Input Interface

This page represents the Symptom Description Interface of the MediLingua system. It allows users to input their health complaints in natural language.

Key Features Shown:

- Multi-Language Support:

The system supports different languages (e.g., NG Pidgin selected), demonstrating inclusivity and accessibility for diverse users.

- Text and Voice Input:

Users can either type their symptoms manually or use the voice input button to speak their symptoms.

- Example Prompts:

The interface provides example symptom descriptions to guide users on how to structure their input for better accuracy.

- Character Counter:

Displays the number of characters used to ensure inputs stay within the allowed limit.

- Health Disclaimer:

A warning message clarifies that the tool is AI-based and does not replace professional medical consultation.

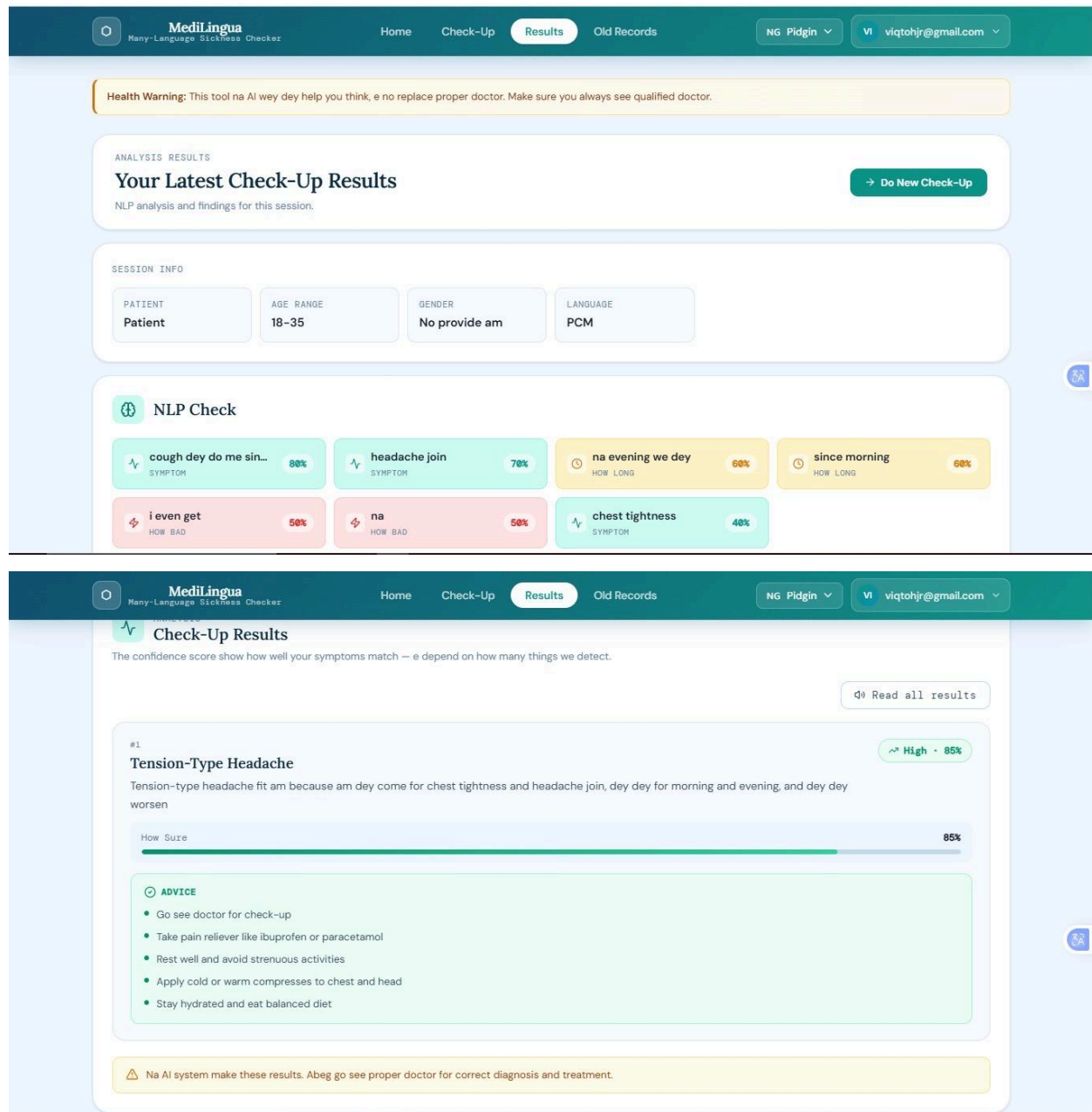
- Processing Indicator:

When the user submits symptoms, the system displays a “Checking...” status, indicating that the AI model is analyzing the input.

System Functionality:

When the user enters symptoms (e.g., cough and headache), the system sends the text to the AI model (Ollama-based model) for

analysis. The model processes the input and prepares a diagnostic response, which will be displayed on the Results page.



1. Results Page Overview

This page presents the outcome of the user's health check-up after symptom analysis. It includes a visible health warning that the AI system does not replace a qualified medical doctor. The session details such as patient category, age range, gender, and selected language are also displayed to provide context for the results.

2. NLP Extraction Results

This section shows how the system used Natural Language Processing (NLP) to extract important information from the user's input. It identifies symptoms, duration, and severity level. Each extracted element is accompanied by a confidence percentage indicating how accurately the system interpreted the input.

3. Predicted Condition Output

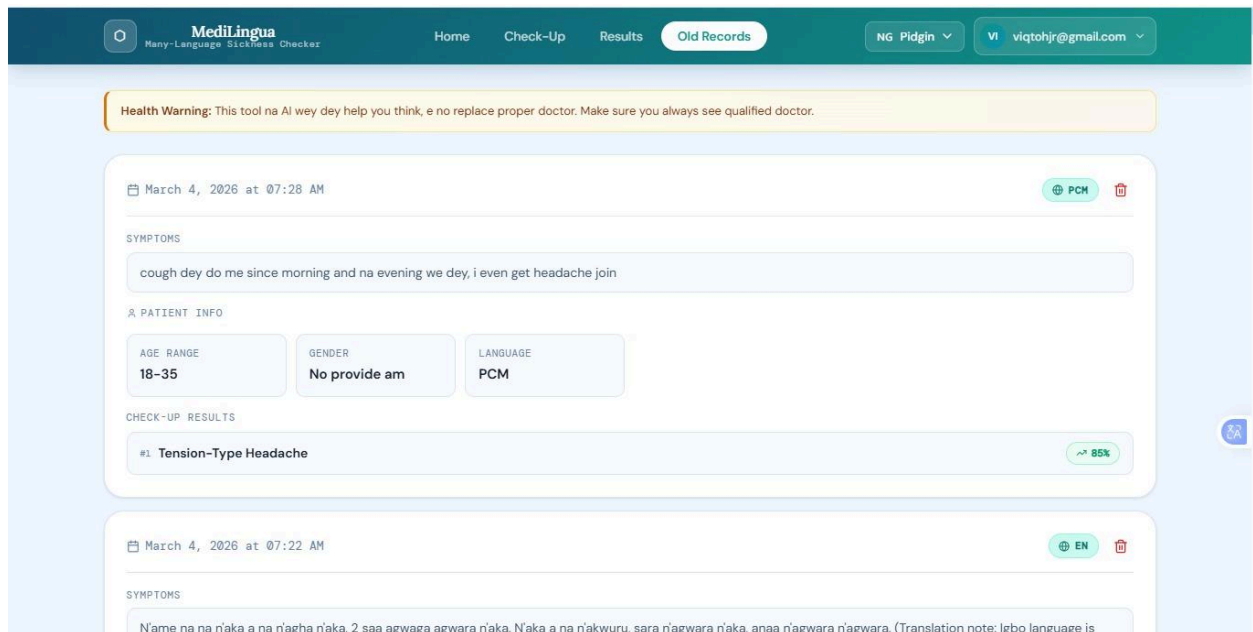
Based on the analyzed symptoms, the system suggests a possible condition. In this case, it predicts Tension-Type Headache with a high confidence score of 85%. A visual progress bar is used to represent the certainty level of the prediction.

4. Medical Advice Output

The system provides supportive recommendations such as visiting a doctor, taking mild pain relievers, resting, applying compresses, and staying hydrated. These suggestions are meant for guidance and not as a confirmed prescription.

5. Disclaimer Notice

A final disclaimer is displayed to remind users that the AI-generated results are not a final diagnosis. Users are strongly encouraged to consult a qualified healthcare professional for accurate diagnosis and treatment.



Page Overview: Old Records

The **Old Records** page serves as a centralized history for users to track their previous health queries. It is designed to provide continuity of care by allowing users to review past symptoms and the AI-generated suggestions they received.

Key Features:

- **Timestamped History:** Each entry is marked with the exact date and time of the check-up, helping users track the duration or frequency of symptoms.
- **Symptom Log:** Displays the exact input provided by the user. As seen in the example, the system preserves the **original language and dialect** (e.g., Nigerian Pidgin), ensuring the patient's voice is accurately recorded.
- **Demographic Context:** Displays the age range and gender provided during the check-up, which are critical factors for medical context.
- **Analysis Results:** Shows the primary suggestion (e.g., "Tension-Type Headache") alongside a **confidence percentage**, indicating how closely the symptoms align with known patterns
- **Multilingual Support Tags:** Clearly labels the language used for the input (e.g., **PCM** for Pidgin, **EN** for English, **IG** for Igbo) to highlight the app's inclusive design.

Note on Safety: Every record is framed by a persistent health warning, reminding users that this history is for informational purposes and does not replace professional medical advice.

11. Challenges and Solutions

During the development of the multilingual symptoms checker, several challenges were encountered:

Challenge 1: Limited Multilingual Medical Datasets

There was difficulty finding structured symptom datasets in multiple languages.

Solution: The team used an English dataset as the base language and implemented translation mechanisms to support additional languages.

Challenge 2: Translation Inconsistencies

Some translated symptom inputs lacked contextual accuracy.

Solution: Input preprocessing and normalization were applied before translation to improve clarity and consistency.

Challenge 3: Model Interpretation Variability

The language model occasionally generated broad or generalized medical responses.

Solution: Prompt structuring and controlled formatting were implemented to guide more relevant outputs.

Challenge 4: System Integration Delays

Integrating translation, NLP processing, and frontend interface initially caused response delays.

Solution: Backend logic was optimized and unnecessary processes were removed to improve performance.

12. Future Work

Although the system achieved its core objectives, several improvements can be made in future versions:

- Add support for more local and international languages.
- Improve model accuracy through structured medical datasets.
- Integrate a medically verified knowledge base for enhanced reliability.

- Improve user interface design for better accessibility and responsiveness.
 - Implement user history tracking for better symptom monitoring.
 - Conduct real-world testing with medical professionals for validation.
-

13. Conclusion

The development of the multilingual symptoms checker using Natural Language Processing successfully addressed the challenge of language barriers in digital health support systems. The system allows users to input symptoms in their preferred language, translates the input for standardized processing, and generates contextual medical insights using a large language model deployed through Ollama.

The project achieved its primary objectives, including multilingual input handling, NLP-based symptom interpretation, and user-friendly interface design. Testing demonstrated consistent system performance and reliable symptom analysis for common health conditions.

Overall, the project was successful in developing a functional and scalable prototype that can serve as a foundation for more advanced multilingual healthcare support systems in the future.

14. References

- Kaggle Symptom Dataset
- Python Official Documentation (<https://docs.python.org>)
- Natural Language Toolkit (NLTK) Documentation
- spaCy NLP Documentation
- Research articles on Natural Language Processing and Healthcare Applications
- Ollama Documentation